

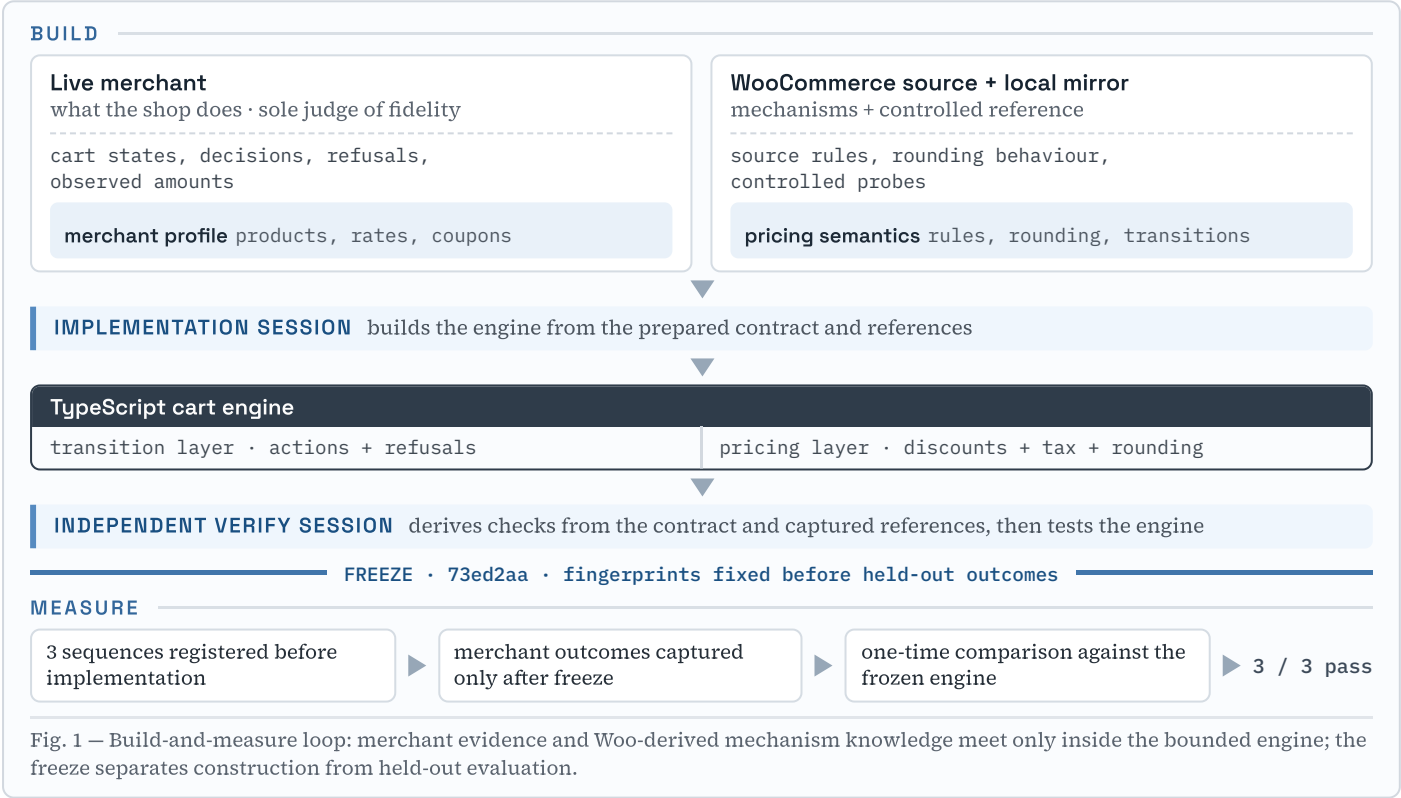
Replicating a live WooCommerce discount engine

Target, slice and reasoning

I chose L’Atelier des Cafés, a live WooCommerce store, and replicated a narrow slice of its cart pricing logic: **item amounts after discounts and taxes for a guest cart in France**, over a small supported catalogue and the merchant’s public coupon. Shipping, fees, gifts and checkout are outside the slice; unsupported inputs are refused rather than approximated.

The cart is stateful without requiring the order lifecycle: add, remove, quantity and coupon actions can change the cart or be refused without mutation. That gives the clone meaningful branching and transitions while keeping it deterministic and resettable.

I considered Medusa and Saleor during target selection, but their headless storefronts made public targets harder to qualify within the search budget. WooCommerce offered a real store, a public promotion and a measurable guest cart quickly. It also gave me a stronger local reference: a pinned vanilla WooCommerce runtime, so merchant/mirror differences have a narrower interpretation than differences between unrelated commerce engines. The live merchant remains the sole judge of merchant fidelity.



Candidate approaches and trade-offs

APPROACH	STRENGTH	MAIN LIMITATION
Behavioural black-box	Measures the merchant directly	Expensive to cover; several rules can explain the same outputs
Inferred cart state machine	Natural fit for transitions and refusals	Weak numeric generalization without a pricing engine
Woo-informed reimplementation + mirror	Merchant evidence + inspectable source + controlled experiments	Requires strict separation between merchant facts and Woo-compatible explanations
API bridge / record-replay	Fast apparent fidelity	Depends on live sessions and does not reconstruct the logic

I chose the third approach, while keeping the useful transition layer from the state-machine option.

What I built

The result is an autonomous TypeScript cart engine. A small merchant profile (“seed”) contains supported products, tax rates, coupon parameters and bounded configuration. Pricing and transition rules live in the engine.

`reset(seed)` creates a clean cart; `dispatch(action)` applies one action and returns an accepted or refused decision; `snapshot()` exposes current lines, coupons and totals. State is in memory only. Refused actions preserve it. A Woo adapter projects merchant or lab responses into the same representation, while an external comparator checks clone outputs against captured references. The engine never reads expected results.

The inferred state machine was the strongest alternative, but transitions alone do not solve numeric behaviour. A controlled two-coupon Woo probe exposed non-trivial rounding between coupon-level and cart-level amounts; reproducing this generally requires pricing semantics underneath the transition layer.

How AI was used

Claude was used mainly to challenge scope, strategy and trade-offs. Codex handled much of the repository work: discovery, preparation of references, implementation and independent verification.

Implementation and verification had different jobs. A fresh verifier derived assertions from the contract and captured evidence before testing the engine, rather than inheriting the implementer’s reasoning. Briefs named authoritative inputs, bounded permitted changes and defined stopping conditions; widening scope or replacing evidence with assumptions required an operator decision.

This caught real problems. Two verification starts were discarded after retrospective material entered the verifier’s context; both stopped before verdict and were restarted cleanly. Separately, continued Woo source review broke an already-green suite with a source-derived rounding regression, exposing an incorrect coupon net/tax split without changing the captured references.

I initially explored stronger per-role sandboxing, then simplified it. The final guarantees are procedural and auditable: fresh sessions, fixed references, Git integrity checks, operator review and held-out merchant outcomes that did not exist during implementation. Prompts, role definitions and selected traces are included in `ai/`.

Verification story

The goal was to make fidelity **falsifiable**.

Merchant discovery supplied the primary references: cart states, decisions, item amounts, coupon amounts and taxes from the live Store API.

The local Woo lab ran a pinned vanilla WooCommerce runtime under a known candidate configuration; the merchant version and customizations remain unknown. On the three merchant states replayed directly against configured vanilla Woo, all retained fields matched. This establishes compatibility on those observations, not the merchant’s full configuration.

The lab also supported targeted probes. A lab-only `LAB20` coupon combined with the merchant-like 10% coupon exposed a non-trivial rounding case: displayed coupon components sum to 285/15 cents of net/tax discount, while the cart reports 284/16. The clone reproduces both. This is Woo lab evidence, not evidence that the merchant supports coupon stacking.

Independent verification targeted correlated errors: a fresh verifier added 38 tests; the final gate reached 180 tests, and a separate evidence audit traced 53 captured decisions, checkpoints and projections back to their sources with no differences. The judge also predated the solution: while the business engine was still a stub, the comparator, replay logic and held-out runner already existed and remained unchanged through official evaluation.

Held-out evaluation tested the frozen engine against merchant outcomes unavailable during development. Three action sequences were registered by hash before implementation; their merchant results were captured only after freeze. All three passed the one-time comparison with no differences, including a quantity refusal reproduced with the same decision, code, message and unchanged cart state.

Three cases are a small sample: they support fidelity within the declared slice, not exhaustive WooCommerce equivalence.

Known and suspected gaps

I did not reconstruct the merchant's full Woo configuration. Product-vs-category coupon restriction, merchant Woo version and exact price-entry convention remain unresolved. Multiple coupons were measured only in the local lab. Shipping, checkout and most of the catalogue are outside the demonstrated domain. Successful coffee pricing above quantity 1 is not demonstrated; captured refusals of larger quantity requests remain part of the demonstrated behaviour.

Next two days and scaling

With two more days, I would test the public coupon on a sale-priced product, then add bounded **differential fuzzing** between the frozen TypeScript engine and the local Woo mirror. That would exercise many valid action sequences more efficiently than adding handwritten carts.

For a larger target, the reusable asset is the **discovery and validation pipeline**: Store API projection, scenario runner, comparator, local lab, profile/seed format, provenance rules and freeze/held-out procedure. Merchant-specific catalogue, tax, coupon and customization rules still require discovery.

I would automate that discovery as an experimental-design loop: choose contrasting products, generate a scenario that separates competing explanations, capture the merchant response, compare with configured Woo, update `observed / compatible / unresolved`, then choose the next experiment for information gain.

If a merchant diverges from core Woo, plugin fingerprints and public documentation can narrow the hypothesis space, but still require validation against the merchant.

Assumptions made

The supported domain is deliberately narrow: guest customer, France, EUR, the prepared products and coupon configuration, and item-level pricing only. Unsupported settings and actions are rejected rather than approximated.

The bounded claim: the engine matches all retained discovery projections and the three preregistered held-out scenarios in the declared FR guest item-pricing domain; a configured vanilla Woo mirror also matched the three directly compared merchant states. This does not establish the merchant's complete configuration or general WooCommerce equivalence.